

Lab C Bonus A

Design goals

- EC2 is private (no public IP)

- No SSH required (use SSM Session Manager)

- Private subnets don't need NAT to talk to AWS control-plane services

- Use VPC Interface Endpoints for:

 - SSM, EC2Messages, SSMMessages (Session Manager)

 - CloudWatch Logs

 - Secrets Manager

 - KMS (optional but realistic)

- Use S3 Gateway Endpoint (common "gotcha" for private environments)

- Tighten IAM: GetSecretValue only for your secret, GetParameter(s) only for your path

Note: If you remove NAT entirely, OS package installs can be tricky unless repos are reachable (often via S3). This skeleton gives you S3 endpoint and leaves NAT as optional "student choice." In many orgs, teams use golden AMIs or image pipelines to avoid yum/apt internet needs in private subnets.

Student verification (CLI) for Bonus-A

- 1) Prove EC2 is private (no public IP)

```
aws ec2 describe-instances \
--instance-ids <INSTANCE_ID> \
--query "Reservations[].Instances[].PublicIpAddress"
```

Expected:

```
null
```

- 2) Prove VPC endpoints exist

```
aws ec2 describe-vpc-endpoints \
--filters "Name=vpc-id,Values=<VPC_ID>" \
--query "VpcEndpoints[].ServiceName"
```

Expected: list includes:

```
ssm
ec2messages
ssmmessages
logs
secretsmanager
s3
```

3) Prove Session Manager path works (no SSH)

```
aws ssm describe-instance-information \
--query "InstanceInformationList[].InstanceId"
```

Expected: your private EC2 instance ID appears

4) Prove the instance can read both config stores

Run from SSM session:

```
aws ssm get-parameter --name /lab/db/endpoint
aws secretsmanager get-secret-value --secret-id <your-secret-name>
```

5) Prove CloudWatch logs delivery path is available via endpoint

```
aws logs describe-log-streams \
--log-group-name /aws/ec2/<prefix>-rds-app
```

How this maps to “real company” practice (short, employer-credible)

Private compute + SSM is standard in regulated orgs and mature cloud shops.

VPC endpoints reduce exposure and dependency on NAT for AWS APIs.

Least privilege is not optional in security interviews.

Terraform submission mirrors how teams ship changes: PR → plan → review → apply → monitor.

Here's your step-by-step guide to implement **Bonus A: Private EC2 + VPC Endpoints** based on your current working Terraform setup.

Overview: What You're Building

You're converting from this:

EC2 (public subnet) → Internet → AWS APIs

To this:

EC2 (private subnet) → VPC Endpoints → AWS APIs (no internet needed)

Step 1: Create Security Group for VPC Endpoints

Add this to your Terraform. VPC Interface Endpoints need a security group that allows HTTPS (443) from your EC2.

```
#####  
# Security Group for VPC Endpoints  
#####  
  
resource "aws_security_group" "Helga_vpce_sg01" {  
  name          = "${local.name_prefix}-vpce-sg01"  
  description   = "Security group for VPC Interface Endpoints"  
  vpc_id        = aws_vpc.Helga_vpc01.id  
  
  ingress {
```

```

    description      = "HTTPS from EC2"
    from_port        = 443
    to_port          = 443
    protocol          = "tcp"
    security_groups   = [aws_security_group.Helga_ec2_sg01.id]
}

egress {
    from_port        = 0
    to_port          = 0
    protocol          = "-1"
    cidr_blocks       = ["0.0.0.0/0"]
}

tags = {
    Name = "${local.name_prefix}-vpce-sg01"
}
}

```

Step 2: Create VPC Interface Endpoints

These replace the need for internet access to reach AWS APIs.

```

#####
# VPC Interface Endpoints (SSM, Logs, Secrets)
#####

# SSM endpoint (core Session Manager)
resource "aws_vpc_endpoint" "Helga_vpce_ssm" {
    vpc_id            = aws_vpc.Helga_vpc01.id
    service_name       = "com.amazonaws.${var.aws_region}.ssm"
    vpc_endpoint_type  = "Interface"
    subnet_ids         = aws_subnet.Helga_private_subnets[*].id
    security_group_ids = [aws_security_group.Helga_vpce_sg01.id]
}

```

```

d]
    private_dns_enabled = true

    tags = {
        Name = "${local.name_prefix}-vpce-ssm"
    }
}

# EC2 Messages endpoint (Session Manager requirement)
resource "aws_vpc_endpoint" "Helga_vpce_ec2messages" {
    vpc_id            = aws_vpc.Helga_vpc01.id
    service_name      = "com.amazonaws.${var.aws_region}.ec2m
essages"
    vpc_endpoint_type = "Interface"
    subnet_ids        = aws_subnet.Helga_private_subnets[*].i
d
    security_group_ids = [aws_security_group.Helga_vpce_sg01.i
d]
    private_dns_enabled = true

    tags = {
        Name = "${local.name_prefix}-vpce-ec2messages"
    }
}

# SSM Messages endpoint (Session Manager requirement)
resource "aws_vpc_endpoint" "Helga_vpce_ssmmessages" {
    vpc_id            = aws_vpc.Helga_vpc01.id
    service_name      = "com.amazonaws.${var.aws_region}.ssmm
essages"
    vpc_endpoint_type = "Interface"
    subnet_ids        = aws_subnet.Helga_private_subnets[*].i
d
    security_group_ids = [aws_security_group.Helga_vpce_sg01.i
d]
    private_dns_enabled = true

```

```

tags = {
    Name = "${local.name_prefix}-vpce-ssmmessages"
}

# CloudWatch Logs endpoint
resource "aws_vpc_endpoint" "Helga_vpce_logs" {
    vpc_id            = aws_vpc.Helga_vpc01.id
    service_name      = "com.amazonaws.${var.aws_region}.logs"
    vpc_endpoint_type = "Interface"
    subnet_ids        = aws_subnet.Helga_private_subnets[*].id
    security_group_ids = [aws_security_group.Helga_vpce_sg01.id]
    private_dns_enabled = true

    tags = {
        Name = "${local.name_prefix}-vpce-logs"
    }
}

# Secrets Manager endpoint
resource "aws_vpc_endpoint" "Helga_vpce_secretsmanager" {
    vpc_id            = aws_vpc.Helga_vpc01.id
    service_name      = "com.amazonaws.${var.aws_region}.secretsmanager"
    vpc_endpoint_type = "Interface"
    subnet_ids        = aws_subnet.Helga_private_subnets[*].id
    security_group_ids = [aws_security_group.Helga_vpce_sg01.id]
    private_dns_enabled = true

    tags = {

```

```

    Name = "${local.name_prefix}-vpce-secretsmanager"
  }
}

```

Step 3: Create S3 Gateway Endpoint

S3 uses a **Gateway** endpoint (free), not Interface.

```

#####
# S3 Gateway Endpoint
#####

resource "aws_vpc_endpoint" "Helga_vpce_s3" {
  vpc_id            = aws_vpc.Helga_vpc01.id
  service_name      = "com.amazonaws.${var.aws_region}.s3"
  vpc_endpoint_type = "Gateway"
  route_table_ids   = [
    aws_route_table.Helga_private_rt01.id,
    aws_route_table.Helga_public_rt01.id
  ]

  tags = {
    Name = "${local.name_prefix}-vpce-s3"
  }
}

```

Step 4: Add Region Variable (if not present)

Add to your `variables.tf`:

```

variable "aws_region" {
  description = "AWS region"
  type        = string
}

```

```
    default      = "us-east-2"
  }
```

Step 5: Move EC2 to Private Subnet

Update your EC2 resource:

```
resource "aws_instance" "Helga_ec201" {
  ami                = var.ec2_ami_id
  instance_type      = var.ec2_instance_type

  # CHANGED: Move to private subnet
  subnet_id          = aws_subnet.Helga_private_subnets
[0].id

  vpc_security_group_ids = [aws_security_group.Helga_ec2_sg0
1.id]
  iam_instance_profile   = aws_iam_instance_profile.Helga_ins
tance_profile01.name
  user_data              = file("${path.module}/user_data.s
h")

  tags = {
    Name = "${local.name_prefix}-ec201"
  }
}
```

Step 6: Remove SSH Ingress from EC2 Security Group

Since you're using Session Manager, SSH is no longer needed:

```
resource "aws_security_group" "Helga_ec2_sg01" {
  name          = "${local.name_prefix}-ec2-sg01"
  description = "EC2 app security group"
```

```

vpc_id      = aws_vpc.Helga_vpc01.id

# REMOVED: SSH ingress rule (no longer needed with Session
Manager)

# Keep HTTP if you still need external access via ALB later
# For now, remove if EC2 is fully private

egress {
  from_port    = 0
  to_port      = 0
  protocol     = "-1"
  cidr_blocks  = ["0.0.0.0/0"]
}

tags = {
  Name = "${local.name_prefix}-ec2-sg01"
}

lifecycle {
  create_before_destroy = true
}
}

```

Step 7: (Optional) Tighten IAM — Least Privilege

Replace the broad `SecretsManagerReadWrite` with a scoped policy:

```

# Replace the SecretsManagerReadWrite attachment with this:
resource "aws_iam_role_policy" "Helga_ec2_secrets_policy" {
  name = "${local.name_prefix}-secrets-policy"
  role = aws_iam_role.Helga_ec2_role01.id

  policy = jsonencode({
    Version = "2012-10-17"

```

```

Statement = [
  {
    Effect = "Allow"
    Action = [
      "secretsmanager:GetSecretValue"
    ]
    Resource = aws_secretsmanager_secret.Helga_db_secret0
1.arn
  },
  {
    Effect = "Allow"
    Action = [
      "ssm:GetParameter",
      "ssm:GetParameters"
    ]
    Resource = "arn:aws:ssm:${var.aws_region}*:parameter/lab/db/*"
  }
]
})
}

```

Then remove the old attachment:

```

# DELETE THIS BLOCK:
# resource "aws_iam_role_policy_attachment" "Helga_ec2_secret
s_attach" { ... }

```

Step 8: Apply Changes

```

terraform plan    # Review the changes
terraform apply   # Apply when satisfied

```

Step 9: Verify (CLI Commands from Bonus A)

1) Confirm EC2 has no public IP:

```
aws ec2 describe-instances \
  --instance-ids <INSTANCE_ID> \
  --query "Reservations[].Instances[].PublicIpAddress"
# Expected: null
```

```
aws ec2 describe-instances \
  --instance-ids i-09bf1685560d0e485 \
  --query "Reservations[].Instances[].PublicIpAddress"
# Expected: null
```

2) Confirm VPC endpoints exist:

```
aws ec2 describe-vpc-endpoints \
  --filters "Name=vpc-id,Values=<VPC_ID>" \
  --query "VpcEndpoints[].ServiceName"
# Expected: ssm, ec2messages, ssmmessages, logs, secretsmanager, s3
```

```
aws ec2 describe-vpc-endpoints \
  --filters "Name=vpc-id,Values=vpc-063d1e6ab946920e6" \
  --query "VpcEndpoints[].ServiceName"
```

3) Confirm Session Manager can reach the private EC2:

```
aws ssm describe-instance-information \
  --query "InstanceInformationList[].InstanceId"
# Expected: your instance ID appears
```

4) From inside Session Manager session, test access:

```
aws ssm get-parameter --name /lab/db/endpoint
aws secretsmanager get-secret-value --secret-id helga/rds/mysql
```

5) Prove CloudWatch Logs delivery path is available via endpoint:

```
aws logs describe-log-streams \
  --log-group-name /aws/ec2/helga-rds-app
# Expected: Returns log streams if the log group exists and the
# endpoint is working.
# If you see streams listed, CloudWatch Logs delivery via endpoint
# is confirmed.
```

Note: If the log group doesn't exist yet, create it first or run your app to generate logs. The key verification is that the CLI command *succeeds* (no timeout/connection errors), proving the VPC endpoint path to CloudWatch Logs is functional.

Manual Log Creation (if log group doesn't exist):

```
# Step 1: Create the log group
aws logs create-log-group \
  --log-group-name /aws/ec2/helga-rds-app

# Step 2: Create a log stream inside the group
aws logs create-log-stream \
  --log-group-name /aws/ec2/helga-rds-app \
  --log-stream-name test-stream

# Step 3: Send a test log event
aws logs put-log-events \
  --log-group-name /aws/ec2/helga-rds-app \
  --log-stream-name test-stream \
  --log-events timestamp=$(date +%s000),message="VPC endpoint
test - $(date)"
```

```
# Step 4: Verify the log was written
aws logs get-log-events \
  --log-group-name /aws/ec2/helga-rds-app \
  --log-stream-name test-stream
# Expected: Your test message appears in the "events" array
```

Success Criteria: If all commands complete without timeout or connection errors, your CloudWatch Logs VPC endpoint is working correctly from the private subnet.

Important Notes

Consideration	What to Know
VPC Endpoint Cost	Interface Endpoints cost ~\$0.01/hour each + data transfer. Budget for ~\$7-10/month per endpoint.
NAT Gateway	You can keep or remove it. If removed, EC2 can't reach <i>any</i> internet — only AWS services via endpoints.
Package Installs	If you remove NAT, <code>yum install</code> won't work unless you use a golden AMI or S3-hosted repos.
App Access	Your app is now private. To expose it externally, you'd need an Application Load Balancer in the public subnet.

Ready to proceed? Start by adding the VPC Endpoint Security Group (Step 1) and the endpoints (Steps 2-3), then `terraform plan` to see what Terraform will create.

```

brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_1 (master)
$ aws ec2 describe-instances \
  --instance-ids i-09bf1685560d0e485 \
  --query "Reservations[].Instances[].PublicIpAddress"
[]

brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_1 (master)
$ aws ec2 describe-vpc-endpoints \
  --filters "Name=vpc-id,Values=vpc-063d1e6ab946920e6" \
  --query "VpcEndpoints[].ServiceName"
[
  "com.amazonaws.us-east-2.s3",
  "com.amazonaws.us-east-2.ssmmessages",
  "com.amazonaws.us-east-2.secretsmanager",
  "com.amazonaws.us-east-2.logs",
  "com.amazonaws.us-east-2.ec2messages",
  "com.amazonaws.us-east-2.ssm"
]

brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_1 (master)
$ aws ssm describe-instance-information \
  --query "InstanceInformationList[].InstanceId"
[
  "i-09bf1685560d0e485"
]

```

```

brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_1 (master)
$ aws ec2 describe-instances \
• --instance-ids i-09bf1685560d0e485 \
  --query "Reservations[].Instances[].PublicIpAddress"
[]

brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_1 (master)
$ aws ec2 describe-vpc-endpoints \
• --filters "Name=vpc-id,Values=vpc-063d1e6ab946920e6" \
  --query "VpcEndpoints[].ServiceName"
[
  "com.amazonaws.us-east-2.s3",
  "com.amazonaws.us-east-2.ssmmessages",
  "com.amazonaws.us-east-2.secretsmanager",
  "com.amazonaws.us-east-2.logs",
  "com.amazonaws.us-east-2.ec2messages",
  "com.amazonaws.us-east-2.ssm"
]

brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_1 (master)
• $ aws ssm describe-instance-information \
  --query "InstanceInformationList[].InstanceId"
[
  "i-09bf1685560d0e485"
]

```

```
sh-5.2$ aws ssm get-parameter --name /lab/db/endpoint
aws secretsmanager get-secret-value --secret-id helga/rds/mysql
{
  "Parameter": {
    "Name": "/lab/db/endpoint",
    "Type": "String",
    "Value": "helga-rds01.chkce02amfxxr.us-east-2.rds.amazonaws.com",
    "Version": 4,
    "LastModifiedDate": "2026-01-12T00:30:48.431000+00:00",
    "ARN": "arn:aws:ssm:us-east-2:919113286081:parameter/lab/db/endpoint",
    "DataType": "text"
  }
}
{
  "ARN": "arn:aws:secretsmanager:us-east-2:919113286081:secret:helga/rds/mysql-JIeYVN",
  "Name": "helga/rds/mysql",
  "VersionId": "terraform-20260112014304575500000001",
  "SecretString": "{\"dbname\":\"t_labdb\",\"host\":\"helga-rds01.chkce02amfxxr.us-east-2.rds.amazonaws.com\",\"password\":\"i8$2\\u003eIu]dQOh60lzzRG#iHJS1mg7\\",
  \"port\":3306,\"username\":\"admin\"}",
  "VersionStages": [
    "AWSCURRENT"
  ],
  "CreateDate": "2026-01-12T01:43:05.519000+00:00"
}
sh-5.2$
```

```
sh-5.2$ aws logs describe-log-streams \
--log-group-name /aws/ec2/helga-rds-app
{
  "logStreams": [
    {
      "logStreamName": "test-stream",
      "creationTime": 1768185568187,
      "firstEventTimestamp": 1768185585000,
      "lastEventTimestamp": 1768185585000,
      "lastIngestionTime": 1768185586183,
      "uploadSequenceToken": "49039859658030226365272888636278404909789273835108161391",
      "arn": "arn:aws:logs:us-east-2:919113286081:log-group:/aws/ec2/helga-rds-app:log-stream:test-stream",
      "storedBytes": 0
    }
  ]
}
sh-5.2$
```

✓ Bonus A: Complete

Completed: January 11, 2026

Lessons Learned

VPC Endpoints

- **Interface vs Gateway:** S3 uses a free Gateway endpoint; all other AWS services (SSM, Logs, Secrets Manager) use Interface endpoints (~\$7-10/month each).
- **Security Group Required:** Interface endpoints need their own security group allowing HTTPS (443) inbound from EC2.
- **Full SSM Stack:** Session Manager requires **three** endpoints: `ssm`, `ec2messages`, and `ssmmessages`. Missing one breaks the connection.

CloudWatch Logs Verification

- **No Log Group = No Test:** You can't verify the endpoint with `describe-log-streams` if the log group doesn't exist yet.
- **Manual Creation Works:** Use `create-log-group` → `create-log-stream` → `put-log-events` → `get-log-events` to prove the path works without waiting for app logs.
- **Success = No Timeout:** The key indicator is that CLI commands complete without connection errors—that proves traffic is flowing through the VPC endpoint.

Private Subnet Gotchas

- **No Internet = No Package Installs:** Without NAT, `yum install` fails. Use golden AMIs or S3-hosted repos.
- **IAM Still Matters:** Even with endpoints, the EC2 role needs correct permissions (`logs:CreateLogGroup` , `logs:PutLogEvents` , etc.).
- **External Access Requires ALB:** A private EC2 can't serve public traffic directly—add an Application Load Balancer in the public subnet if needed.